

FIT5196-S1-2026 Assessment 2

This is a group assessment and is worth 40% of your total mark for FIT5196.

Due date: Thursday 28 May 2026, 11:55pm

Task 1. Data Cleansing (18/40)

For this assessment, you are required to write Python code to analyse your dataset, find and fix the problems in the data. The input and output of this task are shown below:

Table 1. The input and output of Task 1

Input files	Output files	Other Deliverables
Group<group_id>_dirty_data.csv Group<group_id>_missing_data.csv Group<group_id>_outlier_data.csv branches.csv edges.csv nodes.csv	Group<group_id>_dirty_data_solution.csv Group<group_id>_missing_data_solution.csv Group<group_id>_outlier_data_solution.csv	Group<group_id>_ass2_task1.ipynb Group<group_id>_ass2_task1.py

Notes

- All files must be zipped into **Group<group_id>_ass2.zip** (please use **.zip**, not **.rar**, **.7z**, **.tar**, etc.).
- Replace **<group_id>** with your group ID without angle brackets.
- You can find all input files from the folder with your group number [here](#). **Using the wrong input files will result in zero output marks.**
- Strictly follow the instructions in the appendix to generate the **.ipynb** and **.py** files.

Exploring and understanding the data is one of the most important parts of the data wrangling process. You are required to perform graphical and/or non-graphical EDA methods to understand the data first and then find the data problems. In this assessment, you have been provided with three data inputs along with 3 additional files: **branches.csv**, **edges.csv** and **nodes.csv** [here](#). Due to an unexpected scenario, a portion of the data is missing or contains anomalous values. Thus, before moving to the next step in data analysis, you are required to perform the following tasks:

1. **Detect** and **fix** errors in `Group<group_id>_dirty_data.csv`.
2. **Impute** the missing values in `Group<group_id>_missing_data.csv`.
3. **Detect** and **remove** outlier rows in `Group<group_id>_outlier_data.csv` (with respect to the `delivery_fee` attribute only).

Project Background

As a starting point, here is what we know about the dataset in hand: The dataset contains Food Delivery data from a restaurant in Melbourne, Australia¹. The restaurant has three branches around the Melbourne metropolitan area. All three branches share the same menu but they have different management so they operate differently.

Each instance of the data represents a single delivery order. The description of each data column is shown in Table 2.

Table 2. Description of the columns

COLUMN	DESCRIPTION
<code>order_id</code>	A unique id for each order
<code>date</code>	The date the order was made, given in YYYY-MM-DD format
<code>time</code>	The time the order was made, given in hh:mm:ss format
<code>order_type</code>	A categorical attribute representing the different types of orders namely: Breakfast , Lunch or Dinner
<code>branch_code</code>	A categorical attribute representing the branch code in which the order was made. Branch information is given in the <code>branches.csv</code> file.
<code>order_items</code>	A list of tuples representing the order items: the first element of the tuple is the item ordered, and the second element is the quantity ordered for that item.
<code>order_price</code>	A float value representing the order total price
<code>customer_lat</code>	Latitude of the customer coming from the <code>nodes.csv</code> file

¹The dataset is fictional

COLUMN	DESCRIPTION
customer_lon	Longitude of the customer coming from the <code>nodes.csv</code> file
customerHasloyalty?	A logical variable denoting whether the customer has a loyalty card with the restaurant (1 if the customer has loyalty and 0 otherwise)
distance_to_customer_KM	A float representing the shortest distance, in kilometres, between the branch and the customer nodes with respect to the <code>nodes.csv</code> and <code>edges.csv</code> files. Dijkstra algorithm can be used to find the shortest path between two nodes in a graph. Reading materials can be found here .
delivery_fee	A float representing the delivery charges of the order

Notes:

1. The output csv files **must** have the exact same columns as the respective input files. Any misspelling or mismatch will cause the affected file to **fail the auto-marker check and receive 0 marks for that output file** (other files are unaffected).
2. In `Group<group_id>_dirty_data.csv`, any row can carry no more than one anomaly (i.e., there can only be up to one issue in a single row).
3. All anomalies in dirty data have one and only one possible fix.
4. There are no data anomalies in `Group<group_id>_outlier_data.csv` except for outliers. Similarly, there are only coverage data anomalies (i.e., no other data anomalies) in `Group<group_id>_missing_data.csv`.
5. There are three types of meals:
 - Breakfast: served during morning (8am-12pm)
 - Lunch: served during afternoon (12:00:01pm-4pm)
 - Dinner: served during evening (4:00:01pm-8pm)

Each meal has a distinct set of items in the menu (e.g., breakfast items cannot be served during lunch or dinner).

6. In order to get the item unit price, a useful Python package to solve multivariable equations is [numpy.linalg](#).

7. **Delivery fee** is calculated using a different method for each branch. All branches serve Melbourne customers **ONLY**. The fee depends linearly (but in different ways for each branch) on:

1. weekend or weekday (1 or 0)
2. time of day (morning 0, afternoon 1, evening 2)
3. distance between branch and customer

It is recommended to use `sklearn.linear_model.LinearRegression` for solving the linear model as demonstrated in the tutorials. No need to set the variables as categorical variables when modelling; just treat the discrete numbers as continuous variables.

8. Using **proper data** for model training is crucial to have a **good** linear model (i.e., R2 score over 0.95 and very close to 1) to validate the delivery fee. The better your model is, the more accurate your result will be.
9. **If a customer has loyalty, they get a 50% discount on delivery fee**
10. The restaurant uses the [Dijkstra algorithm](#) to calculate the shortest distance between customer and restaurant. Explore the [networkx](#) Python package for this, or find a way to implement the algorithm yourself.
11. The branch and customer nodes are provided in `branches.csv`, `edges.csv` and `nodes.csv`, available [here](#). Schemas:
- `branches.csv` — `branch_code` (str, e.g. NS/TP/BK), `branch_name` (str), `branch_lat` (float), `branch_lon` (float)
 - `nodes.csv` — `node` (int OSM id), `lat` (float), `lon` (float)
 - `edges.csv` — `u` (int node id), `v` (int node id), `distance(m)` (float, edge length to use as the shortest-path weight), `street type` (int), `speed(km/h)` (float)
12. The below columns are error-free (i.e., do not look for any errors in dirty data for them):
- **order_id**
 - **time**
 - **the numeric quantity in order_items**
 - **delivery_fee**
13. For missing data imputation, you are recommended to try all possible methods to impute missing values and keep the most appropriate one that could provide the best performance.

14. As EDA is part of this assessment, no further information will be given publicly regarding the data. However, you can brainstorm with the teaching team during tutorials or on the Ed forum.

15. **No library/package restrictions.**

Methodology (3/18)

The report `Group<group_id>_ass2_task1.ipynb` should demonstrate the methodology (including all steps) to achieve the correct results for all three files.

You need to demonstrate your solution using correct steps.

- Your solution should be presented in a proper way including all required steps.
- You need to select and use the appropriate Python functions for input, process and output.
- Your solution should be efficient, without redundant operations or unnecessary reading and writing of data.

Documentation (1.5/18)

The cleaning task must be explained in a well-formatted report (with appropriate sections and subsections). Please remember that the report must explain the complete EDA to examine the data, your methodology to find the data anomalies and the suggested approach to fix those anomalies.

The report should be organised in a proper structure to present your solutions with clear and meaningful titles for sections and subsections or sub-subsection if needed.

- Each step in your solution should be clearly described and justified. For example, you can explain your idea of the solution, any specific settings, and the reason for using a particular function.
- Explanation of your results, including all intermediate steps, is required. This can help the marking team understand your solution and give partial marks if the final results are not fully correct.
- All your code needs proper (but not excessive) commenting.

Task 2. Data Reshaping (6/40)

You need to complete Task 2 with the `suburb_info.xlsx` file [here](#) **ONLY**. With the given property and suburb-related data, you need to study the effect of different normalisation/transformation methods (e.g., standardisation, min-max normalisation, log, power, and Box-Cox transformation) on these columns: `number_of_houses`, `number_of_units`, `population`,

`aus_born_perc`, `median_income`, and `median_house_price`. You need to observe and explain their effect assuming we want to develop a **linear model** to predict the target `median_house_price` using the five feature attributes above (i.e., all listed columns **except** `median_house_price`, which is the target). Reshaping/transformation may be considered for both the features and the target.

When reshaping the data, we normally have two main criteria.

- First, we want our features to be on the same scale.
- Second, we want our features to have as much linear relationship as possible with the target variable (i.e., `median_house_price`).

You need to first explore the data to see whether scaling or transformation is necessary (if so, why; if not, why not), then perform appropriate actions and document your results and observations. **The aim of this task is to prepare the data for a linear regression model; it is not to build the final linear model.** That is, you need to record every step, beginning with the raw data and ending with any required transformations.

Input files	Submission
Suburb_info.xlsx	Group<group_id>_ass2_task2.ipynb

You could consider Task 2 to be **an open exploratory project**: Jackie and Kiara have received funding to do an exploratory consulting project on the property market. They wish to understand any interesting insights from the relevant features in different suburbs of Melbourne. Before stepping into the final linear regression modelling stage, they want you to prepare the data, determine whether any transformation/normalisation is required, assess whether the data satisfies relevant linear regression assumptions, and explain how the data could be made more suitable for later modelling.

As **an exploratory task**, you **only need** to put your journey of exploration in proper documentation in your `.ipynb` file; **no other output file is required for Task 2**. We will mark based on the `.ipynb` content for Task 2.

Required deliverable in the notebook: a clear, justified **final recommended transformation set** for the 5 feature attributes (and the target `median_house_price`, if you choose to transform it). State explicitly which method you would apply to each column and why, based on the evidence collected during your exploration.

Table 3. Description of the `suburb_info.xlsx` file

COLUMN	DESCRIPTION
suburb	The suburb name, which serves as the index of the data
number_of_houses	The number of houses in the property suburb
number_of_units	The number of units in the property suburb
municipality	The municipality of the property suburb
aus_born_perc	The percentage of the Australian-born population in the property suburb
median_income	The median income of the population in the property suburb
median_house_price	The median house price in the property suburb
population	The population in the property suburb

Documentation (1.5/6)

The reshaping task must be explained in a well-formatted report (with appropriate sections and subsections). Please remember that the report must cover the complete EDA used to examine the data, the rationale for any scaling/transformation choices, and how those decisions improve the suitability of the data for a linear regression model.

The report should be organised in a proper structure to present your solutions with clear and meaningful titles for sections and subsections or sub-subsection if needed.

- Each step in your solution should be clearly described and justified. For example, you can explain your idea of the solution, any specific settings, and the reason for using a particular function.
- Explanation of your results, including all intermediate steps, is required. This can help the marking team understand your solution and give partial marks if the final results are not fully correct.
- All your code needs proper (but not excessive) commenting.

Task 3. Presentation (14/40)

There will be a group presentation to demonstrate your work in A2. The aim of the presentation is to check your understanding of your entire A2 work and to make sure all submissions are

compliant with the academic integrity requirements of Monash. The presentation is a marked component (no hurdle) — your performance will be scored against the rubric.

Task Details:

- **Time/Date:** 15 (Monday) & 16 (Tuesday) June, Week 15.
- **Form:** All members should present one part of their work in person.
- **Duration:** Approximately 25 minutes per group (15 minutes presentation + 10 minutes Q&A)
- **Location:** Room FG01, 02, 03 at Learning Village, 33 Innovation Walk, Clayton.
- **Arrangement:** We will provide a time schedule for every group. Please arrive at your allocated time slot. If you arrive earlier, please wait patiently outside the room.
- **Content:** Demonstrate your solution of A2 tasks.
- **Criterion:** Please refer to A2 marking rubrics.

Requirement:

- Attendance is required to be marked.
- All members need to show and sign the attendance sheet.
- All members need to answer at least one question.

Consequences of non-attendance

Failure to attend the presentation, or inability to satisfactorily demonstrate your work, will result in losing the marks allocated to the presentation component (up to the full 14/40 of Task 3).

The following excuses will not be considered:

- Forgetting to come to the applied session, i.e. Alarm failed, sleepover, missed the bus, etc.
- Forgetting to prepare for the presentation, i.e., forgetting your own solution
- Insufficient time to answer the questions properly, i.e. long silence or thinking time more than 30 sec.
- Being too nervous to understand or talk in English thus not properly answering the questions
- Directly using online resources without proper reference and not understanding the submitted work, i.e. using AI tools to work out the solution without reviewing or understanding

Task 4. Generative AI Tools Declaration Form (required deliverable, 2 marks)

Input files	Submission
Group<group_number>_AI_declaration.docx	Group<group_id>_ass2_task4.pdf Group_AI_Records.docx/.pdf/.txt (if applicable)

Task Details: For this task, all students **must** complete the Generative AI Tools Declaration Form. Use the provided Group<group_number>_AI_declaration.docx template, complete the declaration, and make sure all group members sign the form.

After completing the form, export it as a PDF named Group<group_id>_ass2_task4.pdf and include it in the submission. If your group used Generative AI tools, you must clearly document and **attach all conversation records** with these tools as Group<group_id>_AI_Records.docx/.pdf/.txt.

Requirement:

- This task **must be completed**.
- All group members must sign the declaration form.
- Conversation records should be **all English**.
- **All** conversations need to be recorded. If you are unable to download or use multiple conversations, copy and paste them into **one single file**.
- Links to the chat history are not recommended because of possible unavailability of the links.

Consequences of a missing or incomplete declaration

Failure to submit a complete declaration and **(if applicable) AI conversation records** will be treated as a submission completeness issue and **may trigger an academic integrity review** under the standard Monash process.

Submission Requirements

You need to submit the following **7–8 files** (the 8th, Group<group_id>_AI_Records, is required **only if** your group used Generative AI tools; otherwise submit 7 files):

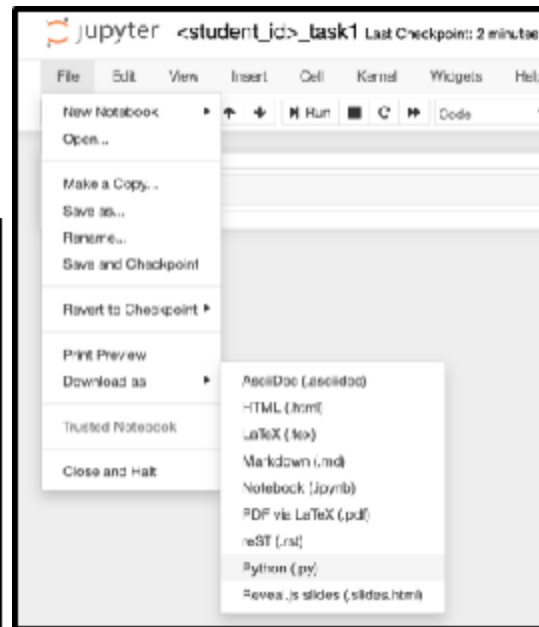
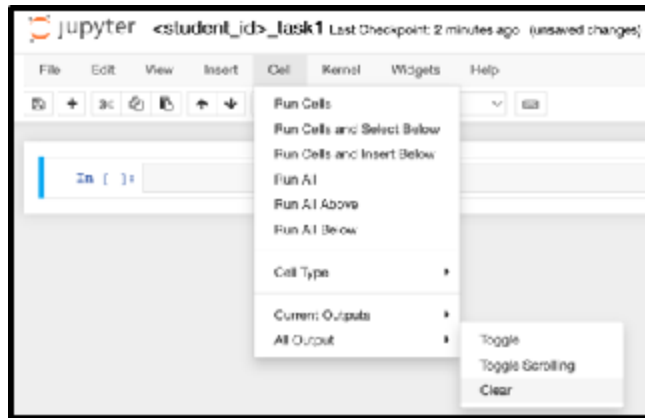
- Group<group_id>_dirty_data_solution.csv: This file should contain the data records with all identified errors fixed.

- `Group<group_id>_missing_data_solution.csv`: This file must contain the data records with all missing values imputed.
- `Group<group_id>_outlier_data_solution.csv`: This file should include the remaining data records with all outliers removed.
- `Group<group_id>_ass2_task1.ipynb`: In this notebook, provide a well-documented report that comprehensively demonstrates your solutions for the dirty, missing, and outlier data files of Assessment 2. You need to clearly present your methodology through a step-by-step process, accompanied by relevant comments and explanations. Your approach can vary, but clarity is paramount. Please keep this notebook easy to read, as you will lose marks if we cannot understand it. **Make sure the cell outputs are not cleared.**
- `Group<group_id>_ass2_task1.py`: This file will be used for plagiarism checking. Make sure the cell outputs are cleared before exporting.
- `Group<group_id>_ass2_task2.ipynb`: In this notebook, provide a well-documented report that comprehensively demonstrates your solutions for Task 2. You need to clearly present your investigation, your EDA and other methodology through a well-organised reporting format, accompanied by relevant justifications, explanations and references (if any). As an open exploratory task, we expect to see you reach out to check the dataset, model assumptions and relevant research to enrich the report. Please keep this notebook easy to read, as you will lose marks if we cannot understand it. **Make sure the cell outputs are not cleared.**
- `Group<group_id>_ass2_task4.pdf`: This is the declaration form you need to submit.
- `Group<group_id>_AI_Records.docx / .pdf / .txt` (if applicable): This is the AI communication history if you declare that you used generative AI in your A2. Any one of the three formats is acceptable.

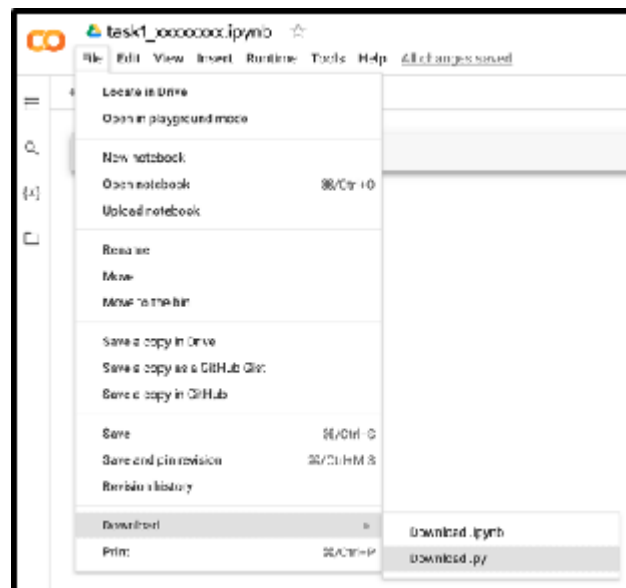
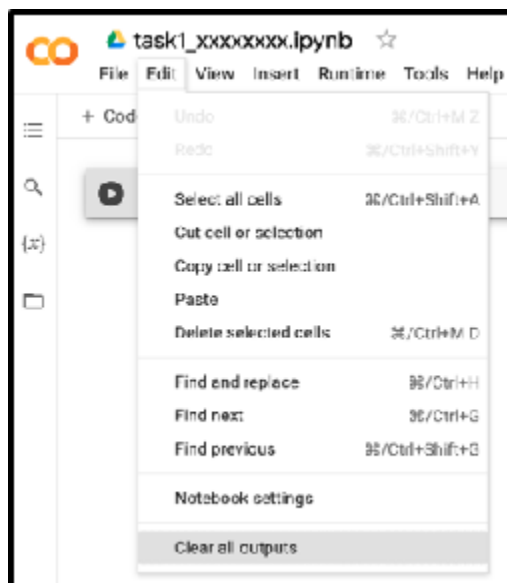
Appendix

To generate a `.py` file, you need to clear all the cell outputs, and then download it.

In Jupyter Notebook:



In Google Colab:



Submission Checklist

- Zip all the submission files for Assessment 2 into a single file named `Group<group_id>_ass2.zip` (any other format, e.g., `.rar` or `.7z`, will be penalised).
- There are **7–8 files** in your compressed zip file (8 only if `AI_Records` is included).
- Replace `<group_id>` with your group ID without angle brackets. Use zero padding, e.g., `001`, `010`.
- Make sure **both** members of your group click the **Submit** button on Moodle.
- Strictly follow the file naming standard. Any misnamed file will carry a penalty.
- Make sure that your `.ipynb` files contain printed output, while your `.py` file does not include any output.
- Ensure that all your files are parsable and readable. You can achieve this by re-reading all your generated files back into Python (e.g., using `read_csv` for CSV files). These checks are only sanity checks and hence should not be added to your final submission.
- Provide the declaration form and attend your allocated applied session in Week 12. The declaration is a required deliverable; the presentation contributes the Task 3 marks.

Note: All submissions will be put through a plagiarism detection software which automatically checks for their similarity with respect to other submissions. Any plagiarism found will trigger the Faculty's relevant procedures and may result in severe penalties, up to and including exclusion from the university.